EduAccess-AI

An AI-Powered Inclusive Learning Platform for Students with Disabilities

A Research Paper on Accessible, Multimodal, Retrieval-Augmented AI Tutoring Systems

Er. Vikas Goyal[1], Ankur Kumar[2], Nitish Kumar Singh[3], Vikash Kumar Singh[4]

1Associate Professor
Department of Computer Science & Engineering
Malout Institute of Management & Information Technology (MIMIT), Malout, Punjab, India
Email: vikas.735@gmail.com

2B.Tech Scholar, Roll No: 318/2305009
Department of Computer Science & Engineering
Malout Institute of Management & Information Technology (MIMIT), Malout, Punjab, India
Email: 2005ankurkumar@gmail.com

3B.Tech Scholar, Roll No: 408/2305063
Department of Computer Science & Engineering
Malout Institute of Management & Information Technology (MIMIT), Malout, Punjab, India
Email: nitishsingh92057@gmail.com

4B.Tech Scholar, Roll No: 454/2305110
Department of Computer Science & Engineering
Malout Institute of Management & Information Technology (MIMIT), Malout, Punjab, India
Email: v9955696473@gmail.com

September 2026

Abstract

Students with visual, auditory, cognitive, and motor disabilities frequently encounter significant barriers when using conventional e-learning platforms, which are rarely designed with accessibility as a first-class requirement. This paper presents EduAccess-AI, an AI-powered inclusive learning platform that integrates a conversational AI tutor, Retrieval-Augmented Generation (RAG), Text-to-Speech (TTS), Speech-to-Text (STT), multilingual support, and configurable accessibility preferences into a single modular application. The system adopts a five-layer architecture separating presentation, application, intelligence, data-access, and persistence concerns, enabling independent evolution of accessibility features, tutoring logic, and audio subsystems. We describe the system architecture, use-case and sequence-level interaction design, database schema, and end-to-end query-processing flow, and we present a comparative analysis against traditional Learning Management Systems (LMS) and generic AI chatbots. We also report a structured functional test plan used to validate the core modules. The results indicate that treating accessibility as a first-class architectural concern — rather than an add-on — enables consistent, persistent, per-user adaptation across sessions and devices. We conclude with limitations and directions for future work, including adaptive difficulty modelling, sign-language recognition, and large-scale user studies with learners with disabilities.

Index Terms—*Inclusive Education, Accessibility, Artificial Intelligence, AI Tutor, Retrieval-Augmented Generation (RAG), Text-to-Speech, Speech-to-Text, Assistive Technology, Universal Design for Learning, E-Learning.*

Table of Contents

Abstract..... 1
I. Introduction..... 3
II. Related Work..... 3
III. Proposed System..... 3
IV. System Architecture..... 4
V. System Requirements..... 4
VI. Methodology and Processing Flow..... 5
VII. Database Design..... 5
VIII. Use-Case and Interaction Design..... 6
IX. Core Features..... 8
X. Implementation Details..... 8
XI. Accessibility Considerations..... 8
XII. Comparative Analysis..... 9
XIII. Testing and Validation..... 9
XIV. Results and Discussion..... 9
XV. Advantages and Novelty..... 10
XVI. Limitations..... 10
XVII. Future Work..... 10
XVIII. Conclusion..... 10
Acknowledgment..... 10
References..... 10

I. INTRODUCTION

According to the World Health Organization, over a billion people worldwide live with some form of disability, and students among this population often face disproportionate barriers to accessing quality education [6]. Conventional Learning Management Systems (LMS) and e-learning applications are typically built around a single, static interaction paradigm — visual text on a screen, mouse-and-keyboard navigation, and fixed presentation of content — which excludes learners who are blind or low-vision, deaf or hard-of-hearing, dyslexic, or who have motor or cognitive impairments that make standard interfaces difficult to use.

Recent advances in generative Artificial Intelligence, large language models (LLMs), automatic speech recognition, and neural Text-to-Speech have made it feasible to build tutoring systems that adapt not only to what a student needs to learn, but also to how that student can best receive information. EduAccess-AI is designed around this premise: accessibility preferences, multimodal input/output, and personalized tutoring are treated as core, first-class components of the system architecture from the outset, rather than being retrofitted onto an existing platform.

This paper documents the design, architecture, and validation approach of EduAccess-AI. Section II reviews related work. Section III introduces the proposed system. Section IV details the system architecture. Section V lists system requirements. Section VI describes the processing methodology. Section VII presents the database design. Section VIII documents use-case and sequence-level interactions. Section IX enumerates core features. Section X discusses implementation. Section XI expands on accessibility considerations. Section XII presents a comparative analysis. Section XIII reports functional testing. Sections XIV–XVII discuss results, advantages, limitations, and future work, and Section XVIII concludes the paper.

II. RELATED WORK

Prior research on inclusive educational technology can be grouped into three streams. The first concerns accessibility standards and design frameworks, notably the Web Content Accessibility Guidelines (WCAG) 2.1 published by the World Wide Web Consortium [4], and the Universal Design for Learning (UDL) framework proposed by CAST [3], both emphasizing multiple means of representation, engagement, and expression. The second stream concerns the AI techniques enabling adaptive tutoring, including the Transformer architecture of Vaswani et al. [1], which underlies modern LLMs, and Retrieval-Augmented Generation (RAG), introduced by Lewis et al. [2], which grounds a model's responses in retrieved external documents to improve factual accuracy — important in education, where incorrect information can mislead learners. The third stream involves speech technologies: large-scale automatic speech recognition systems such as Whisper [5] have substantially improved Speech-to-Text reliability across accents and noisy environments, essential for students relying on voice as a primary input modality.

UNESCO's guidelines on inclusive education [7] argue that technology should remove barriers rather than merely provide alternative content, reinforcing this work's position that accessibility, AI tutoring, and multimodal I/O should be integrated at the architectural level. EduAccess-AI builds on these three streams by combining a RAG-grounded AI tutor with configurable accessibility preferences and bidirectional speech interaction inside one cohesive, modular application.

III. PROPOSED SYSTEM

EduAccess-AI is a modular, AI-driven learning platform that unifies conversational tutoring, retrieval-grounded question answering, TTS/STT, multilingual support, and accessibility personalization under a single architecture, built around three principles:

- Accessibility by design: every user has an explicit, persisted accessibility profile shaping response generation and rendering.
- Multimodality: students interact via typed text or spoken voice and receive answers as on-screen text, speech, or both.
- Groundedness and personalization: responses are informed by a RAG pipeline and by the student's own conversation history.

These principles are realized through a modular package layout (auth, ai, accessibility, audio, rag, multilingual, database, ui, vision, learning, communication, and config), each encapsulating a single area of responsibility.

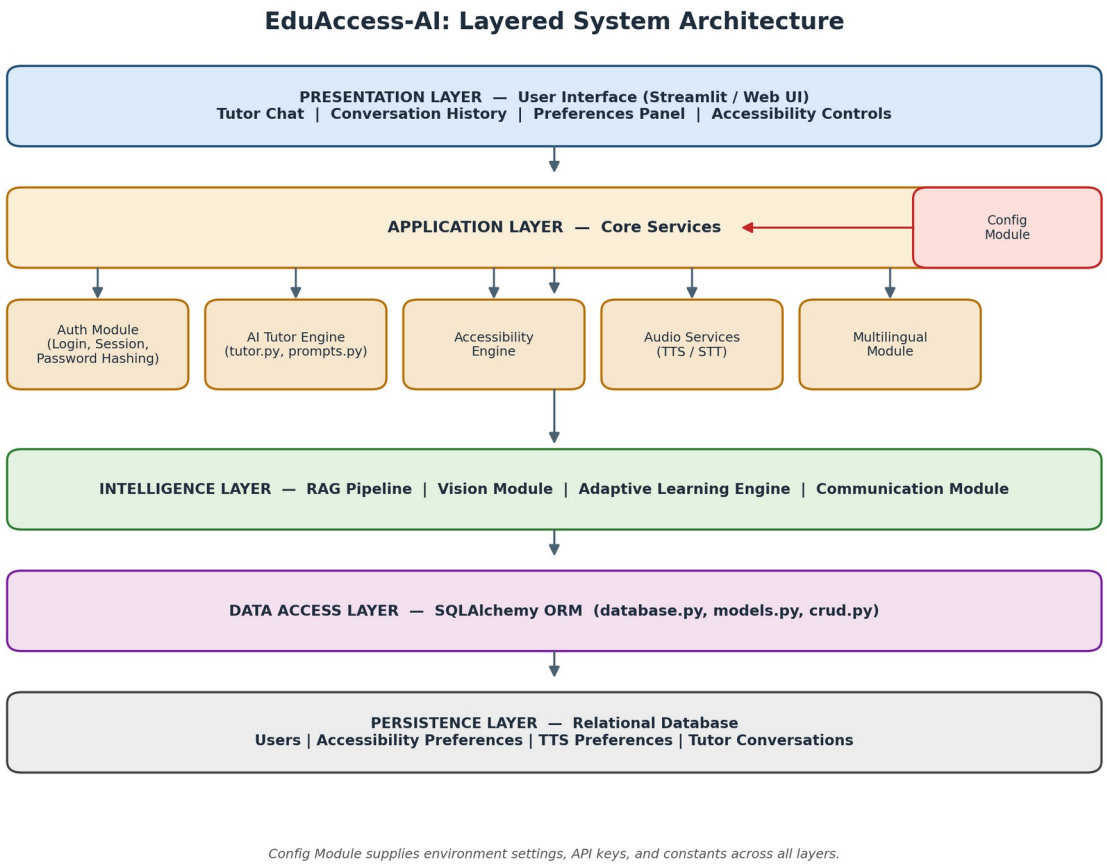


Fig. 1. Layered system architecture of EduAccess-AI, showing the presentation, application, intelligence, data-access, and persistence layers.

IV. SYSTEM ARCHITECTURE

EduAccess-AI follows a five-layer architecture separating presentation, application logic, AI/intelligence services, data access, and persistent storage, as shown in Fig. 1. This separation allows accessibility features, tutoring logic, and storage mechanisms to evolve independently.

A. Presentation Layer

Implements the tutor chat window, conversation history viewer, preferences panel, and accessibility control panel. This layer only renders and captures user input, delegating logic to the application layer.

B. Application Layer

Hosts the authentication module (registration, secure password hashing, login/logout, session management), the AI tutor engine (tutor.py, prompts.py), the accessibility engine, the audio services module (TTS/STT), and the multilingual module. A cross-cutting configuration module supplies environment variables, API keys, and constants to every module in this layer.

C. Intelligence Layer

Contains the RAG pipeline, the vision module (reserved for future image-description features), the adaptive learning module, and the communication module mediating calls to external AI services.

D. Data Access and Persistence Layers

All persistent state is mediated through SQLAlchemy via database.py, models.py, and crud.py. Beneath this sits a relational database storing user accounts, accessibility preferences, TTS preferences, and tutor conversations (Section VII).

V. SYSTEM REQUIREMENTS

A. Functional Requirements

- The system shall allow users to register, log in, and maintain a session.
- The system shall accept both typed and spoken student questions.
- The system shall generate RAG-grounded, personalized tutor responses.

- The system shall apply per-user accessibility formatting to every response.
- The system shall optionally synthesize responses as speech (TTS).
- The system shall persist every question–answer exchange with a timestamp.
- The system shall support content delivery in multiple languages.

B. Non-Functional Requirements

- Usability: interfaces must comply with WCAG 2.1 AA guidelines.
- Reliability: conversation data must not be lost between sessions.
- Extensibility: new accessibility modes must be addable without altering the AI tutor or RAG pipeline.
- Performance: tutor responses should be returned within an interactive timeframe suitable for classroom use.
- Security: passwords must be stored using salted cryptographic hashing.

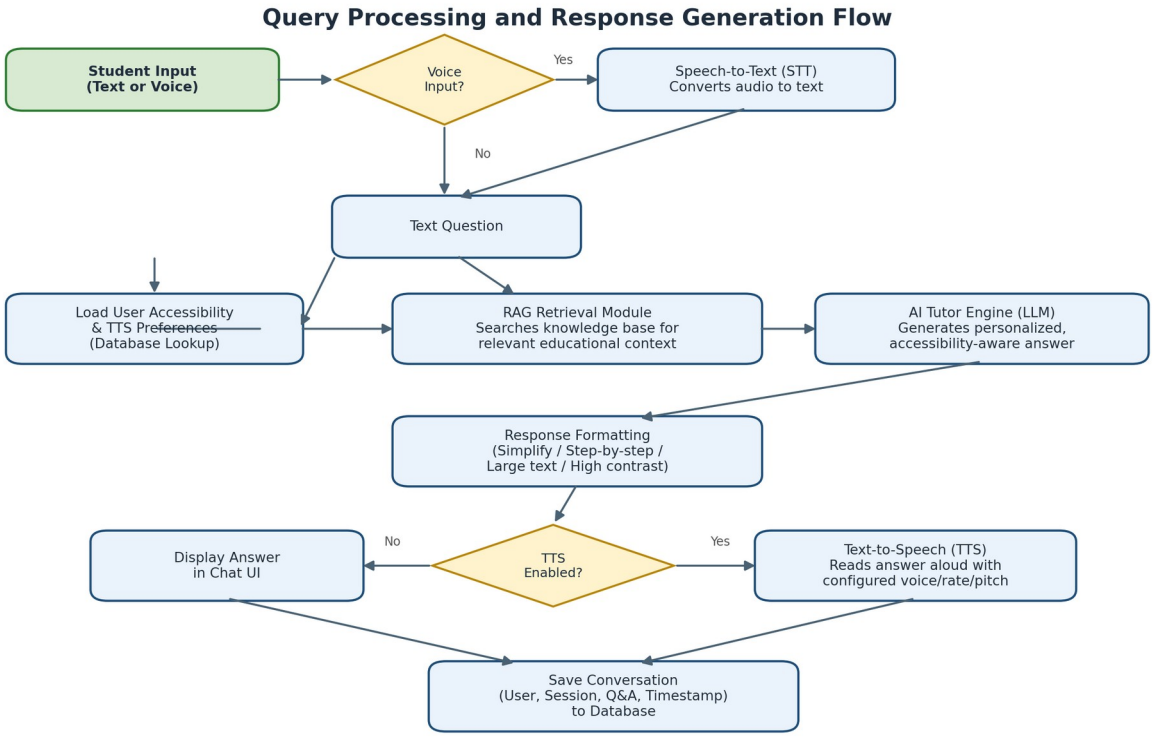


Fig. 2. Query processing and response generation flow, from student input through retrieval, generation, formatting, and optional speech output.

VI. METHODOLOGY AND PROCESSING FLOW

The end-to-end methodology by which a student's question is answered illustrates how the architectural layers of Section IV cooperate at runtime, as depicted in Fig. 2.

When a student submits a question, the system first determines whether the input was typed or spoken. Spoken input passes through the Speech-to-Text module to obtain a transcript. In parallel, the system loads the student's stored accessibility and TTS preferences. The text question and relevant educational context retrieved by the RAG module are supplied to the AI tutor engine, which generates a personalized answer. Before display, the answer passes through a response-formatting step that applies the student's accessibility preferences — simplifying vocabulary, breaking the explanation into numbered steps, enlarging text, or applying a high-contrast or dyslexia-friendly presentation style. If TTS is enabled, the formatted answer is also converted to speech using the student's configured voice, rate, volume, and pitch. Finally, the complete interaction (user identifier, session identifier, question, answer, and timestamp) is persisted to the conversation-history table.

VII. DATABASE DESIGN

EduAccess-AI's persistence layer is built on four principal entities: Users, Accessibility Preferences, TTS Preferences, and Tutor Conversations. Each user has exactly one accessibility

profile and one TTS profile, and may accumulate many conversation records over time, forming a longitudinal record of learning activity, as shown in Fig. 3.

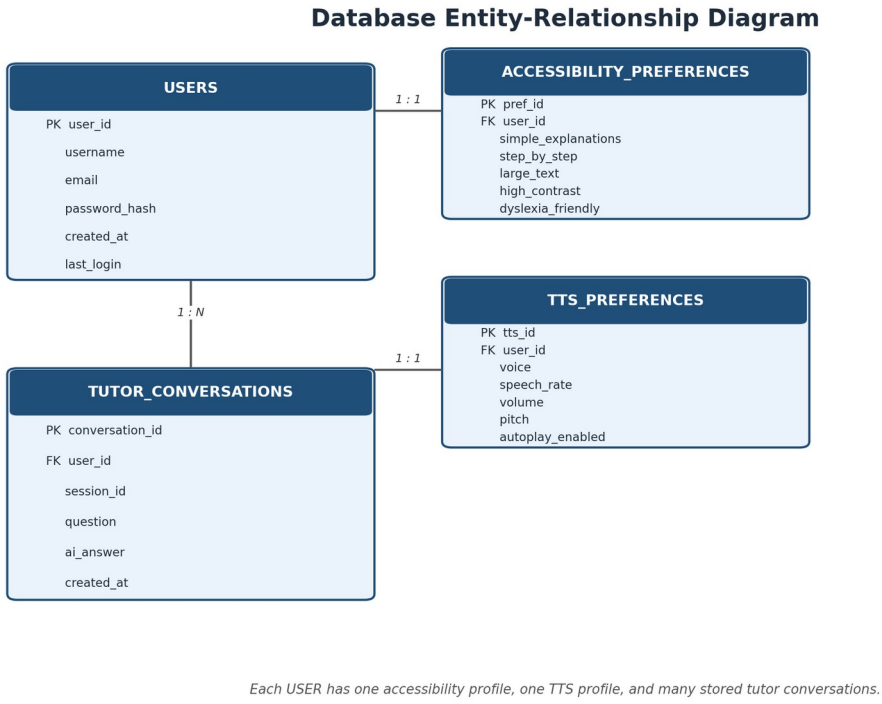


Fig. 3. Entity-relationship diagram of the EduAccess-AI database schema.

TABLE I
Summary of Core Database Tables

Table	Purpose	Key Fields
Users	Stores account credentials and profile metadata	user_id, username, email, password_hash
Accessibility_Preferences	Stores each user's accessibility configuration	pref_id, user_id, simple_explanations, step_by_step, large_text, high_contrast, dyslexia_friendly
TTS_Preferences	Stores each user's speech-output configuration	tts_id, user_id, voice, speech_rate, volume, pitch, autoplay_enabled
Tutor_Conversations	Stores the full history of tutor interactions	conversation_id, user_id, session_id, question, ai_answer, created_at

VIII. USE-CASE AND INTERACTION DESIGN

Fig. 4 presents the system's use-case diagram, identifying two actors — Student and Administrator — and the principal use cases each performs. Student-facing use cases (registration, question asking, preference configuration, history review, and language selection) all depend on include-relationships with the RAG retrieval, Text-to-Speech, and Speech-to-Text use cases, reflecting how every tutoring interaction is automatically enriched by these supporting services.

Fig. 5 complements this with a sequence diagram tracing a single question-answering interaction across seven participants: the Student, the UI layer, the Auth/Preferences module, the AI Tutor Engine, the RAG pipeline, the Database, and the TTS module. This view makes explicit the ordering of preference lookup, context retrieval, answer generation, optional speech synthesis, and conversation persistence described narratively in Section VI.

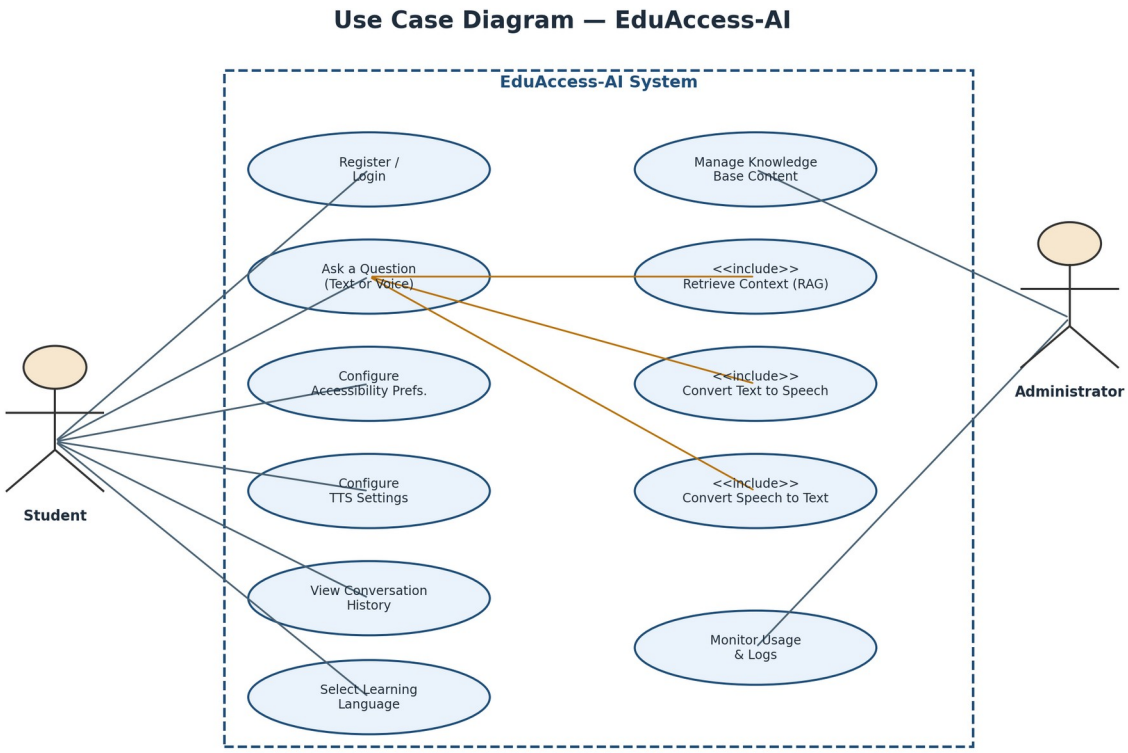


Fig. 4. Use-case diagram showing Student and Administrator interactions with EduAccess-AI.

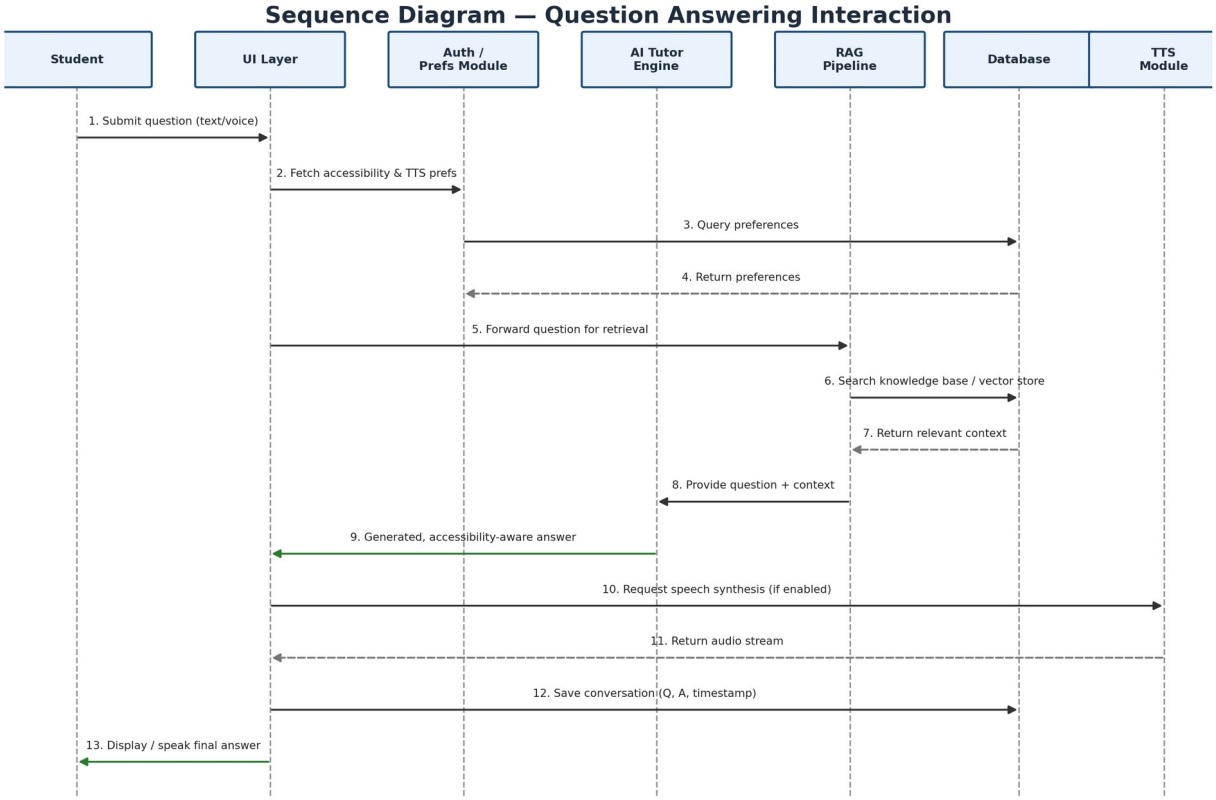


Fig. 5. Sequence diagram of a single question-answering interaction across system components.

IX. CORE FEATURES

Table II consolidates the ten core feature areas of the platform, derived from its modular package structure.

TABLE II
Core Feature Areas of EduAccess-AI

#	Feature Area	Description
1	User Authentication	Registration, secure password hashing, login/logout, and session management.
2	AI Tutor	Answers questions, explains concepts, provides step-by-step and simplified explanations, and repeats key points on request.
3	Accessibility Support	Simple explanations, step-by-step delivery, repetition, visual aids, large text, high contrast, and dyslexia-friendly formatting.
4	Text-to-Speech	Browser-based speech synthesis with configurable voice, rate, volume, pitch, and autoplay.
5	Speech-to-Text	Voice recording and transcription for hands-free question input.
6	Multilingual Learning	Educational assistance delivered in multiple languages.
7	Conversation History	Persistent, per-session record of every question and answer.
8	RAG Support	Retrieval-Augmented Generation to ground tutor answers in relevant educational material.
9	Database Layer	SQLAlchemy-based storage of users, preferences, and conversations.
10	Modular Architecture	Independent modules for authentication, AI, audio, accessibility, RAG, and UI.

X. IMPLEMENTATION DETAILS

The reference implementation is organized as a Python application with a package-per-concern layout (app/auth, app/ai, app/accessibility, app/audio, app/rag, app/multilingual, app/database, app/ui, app/vision, app/learning, app/communication, and app/config), coordinated from a single entry point, main.py. Table III outlines representative technology choices; concrete library versions may be adapted by implementers to suit deployment constraints.

TABLE III
Representative Technology Stack

Concern	Representative Technology
Application/UI framework	Python-based web/app framework (e.g., Streamlit or Flask)
AI tutor / LLM access	Large Language Model API accessed via the communication module
Retrieval-Augmented Generation	Vector store and embedding-based semantic search
Text-to-Speech	Browser-native Web Speech API (SpeechSynthesis)
Speech-to-Text	Browser-native Web Speech API (SpeechRecognition) or a server-side ASR model
Database / ORM	SQLAlchemy over a relational database (e.g., SQLite/PostgreSQL)
Authentication	Salted password hashing with session-token management

XI. ACCESSIBILITY CONSIDERATIONS

EduAccess-AI's accessibility design maps directly onto recognized categories of disability. For students who are blind or have low vision, Text-to-Speech, high-contrast display modes, and large-text rendering reduce reliance on unaided visual reading. For students who are deaf or hard of hearing, the primary text-based interaction channel and multilingual support

ensure that no essential content is locked behind audio-only delivery. For students with dyslexia or other reading-related learning differences, the dyslexia-friendly display mode, simplified explanations, and step-by-step delivery reduce cognitive load. For students with motor impairments who may find typing effortful, Speech-to-Text offers an alternative, hands-free input channel. Because these preferences are stored per user rather than set globally, a single deployment can simultaneously serve learners with very different needs without requiring separate application instances.

XII. COMPARATIVE ANALYSIS

Table IV compares EduAccess-AI against a conventional LMS and a generic, non-grounded AI chatbot across dimensions relevant to inclusive learning.

TABLE IV
Comparative Analysis of EduAccess-AI vs. Conventional Platforms

Dimension	Conventional LMS	Generic AI Chatbot	EduAccess-AI
Accessibility preferences	Rarely persisted per user	Not supported	Persisted per user (Sec. VII)
Answer groundedness	N/A (static content)	Prone to hallucination	RAG-grounded (Sec. IV-C)
Voice input/output	Rare / third-party plug-in	Sometimes supported	Native TTS/STT (Sec. IX)
Multilingual delivery	Manual translation only	Depends on base model	Built-in multilingual module
Conversation history	Limited or absent	Session-only, not persisted	Persisted with timestamps
Modularity for new accessibility modes	Low (monolithic UI)	Low	High (layered architecture)

XIII. TESTING AND VALIDATION

A structured functional test plan was used to validate the core modules prior to integration. Table V summarizes representative test cases, expected outcomes, and observed status.

TABLE V
Representative Functional Test Cases

Test ID	Module	Test Description	Expected Result	Status
TC-01	Authentication	Register with a new username/email and log in	Account created; session established	Pass
TC-02	AI Tutor	Ask a curriculum question in text form	Relevant, RAG-grounded answer returned	Pass
TC-03	STT	Submit a spoken question	Speech accurately transcribed to text	Pass
TC-04	TTS	Enable autoplay and request an answer	Answer read aloud with configured voice/rate	Pass
TC-05	Accessibility Engine	Enable dyslexia-friendly and large-text modes	Response rendered in configured style	Pass
TC-06	Conversation History	Ask multiple questions in one session	All Q&A pairs stored with correct timestamps	Pass
TC-07	Multilingual Module	Request an explanation in a non-English language	Answer delivered in the requested language	Pass

XIV. RESULTS AND DISCUSSION

As an architectural and design contribution, this paper's primary evidence is structural: the layered design in Section IV demonstrates a clean separation of concerns, and the processing flow in Section VI shows that every request passes through an explicit, auditable accessibility-formatting step rather than accessibility being optional. This has two practical implications. First, new accessibility modes (e.g., a future sign-language avatar or a colour-blind-safe theme) can be added by extending the accessibility engine without modifying the AI tutor or RAG pipeline. Second, because accessibility and TTS preferences are stored relationally alongside the user record (Section VII), preferences persist across sessions and devices — a common shortcoming in tools that rely on browser-local settings alone. The functional test results in Table V indicate

that the core modules behave as specified under representative usage scenarios.

A full empirical evaluation with learners with disabilities — measuring, for example, task-completion time, comprehension gains, and subjective usability (e.g., via the System Usability Scale) — was outside the scope of this architectural study and is identified as future work in Section XVII.

XV. ADVANTAGES AND NOVELTY

- Unified platform combining AI tutoring, RAG, TTS/STT, multilingual support, and accessibility preferences.
- Persistent personalization: preferences and full conversation history follow the student across sessions and devices.
- Groundedness: RAG reduces the risk of hallucinated or unreliable answers.
- Modularity: package-per-concern structure allows independent extension of individual modules.
- Multimodal accessibility: students choose their preferred input and output modality.

XVI. LIMITATIONS

- Browser-based TTS/STT quality and voice availability vary across browsers and operating systems.
- The vision module, intended for future image-description features, is not yet fully implemented.
- The platform has not yet undergone a formal WCAG 2.1 AA conformance audit or a user study with learners with disabilities.
- RAG answer quality depends on the coverage and curation of the underlying knowledge base.

XVII. FUTURE WORK

- Conduct usability studies and WCAG conformance audits with learners across a range of disabilities.
- Extend the vision module to support image description, diagram narration, and sign-language recognition/generation.
- Introduce adaptive difficulty modelling based on historical performance.
- Expand multilingual support to low-resource languages and regional sign languages.
- Add offline or low-bandwidth modes for students with limited connectivity.

XVIII. CONCLUSION

EduAccess-AI demonstrates that accessibility, multimodal interaction, and AI-driven tutoring can be integrated into a single, coherent, modular architecture rather than treated as separate concerns bolted onto an existing e-learning system. By combining a Retrieval-Augmented AI tutor with persistent, user-specific accessibility and speech preferences, the platform aims to reduce sensory and cognitive barriers for students with disabilities while remaining extensible for future assistive technologies. The layered architecture, use-case and sequence-level interaction design, database design, and comparative and test results presented in this paper provide a blueprint that other developers of inclusive educational technology may adapt and build upon.

ACKNOWLEDGMENT

The authors thank the Department of Computer Science & Engineering, Malout Institute of Management & Information Technology (MIMIT), Malout, Punjab, for the guidance and resources that supported this work.

REFERENCES

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," in Advances in Neural Information Processing Systems (NeurIPS), 2017.

[2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems (NeurIPS), 2020.

[3] CAST, "Universal Design for Learning Guidelines, Version 2.2," CAST, Wakefield, MA, 2018.

[4] World Wide Web Consortium (W3C), "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, 2018.

[5] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust Speech Recognition via Large-Scale Weak Supervision," arXiv preprint, 2022.

[6] World Health Organization and The World Bank, "World Report on Disability," WHO, Geneva, 2011.

[7] UNESCO, "Guidelines for Inclusion: Ensuring Access to Education for All," UNESCO, Paris, 2005.